

TanStack Trust: A Base for Domain-Owned Software Guarantees

Subtitle: Separating domain meaning, reusable evidence mechanics, and consumer policy

Status: Draft RFC for feedback

Audience: TanStack maintainers and contributors building libraries, domain packages, and agent-facing developer tools

Abstract

Coding is becoming cheap. Justified reliance on the resulting software is not. An agent can produce an implementation quickly. Somebody still has to say what the software must guarantee, decide which checks are meaningful, establish that those checks reached the behavior they claim to test, preserve failures, and determine what the available evidence permits.

This RFC proposes **TanStack Trust**, a small evidence-mechanics layer for building those systems inside specific software domains. TanStack Trust does not decide whether an agent, change, or application is trustworthy in the abstract. A domain package owns the meaning of its guarantees, its rules and checks, the evidence it accepts, and the contexts where that evidence applies. TanStack Trust owns the reusable mechanics for recording observations, evaluating declared support routes, tracking freshness and contradictions, retaining repair history, and exposing that state through common interfaces. Consumer policy owns permission, severity, fallback, and deployment action.

For the foreseeable future, Trust will be built and shipped as part of TanStack DB Endpoints. It is an internal architectural layer and evidence contract, not a separate product or package roadmap. “Small” describes the authority the layer is allowed to exercise, not a promise about its eventual line count or operational footprint. Keeping Trust distinct from Endpoints domain semantics means maintaining clean ownership, module, test, status, and interface seams inside one product. Extraction remains an optional later decision.

The current Endpoints worktree implements a finite local Trust kernel, verified file storage, one Endpoints check, and CLI, LSP, and MCP adapters over one service. That is a useful vertical slice. It is not evidence that the architecture spans every software domain, that its registered rules are sound, or that agent-produced software is now safe to ship.

In this RFC, a **guarantee** is a bounded domain claim with declared evidence, conditions, and limits. The word does not imply a mathematical proof, universal coverage, certification, or permission to act unless the domain and consumer contracts explicitly supply those things.

How to read the status labels

Completeness is the engineering goal, so the RFC must distinguish what exists from what the complete system should eventually express.

Label	Meaning
Implemented	Present in the inspected local prototype.
Bounded evidence	Observed or tested only inside the named finite domain.
Proposed contract	Part of the architecture described here but not yet a stable public API.
Normative guidance	A rule for authoring or evaluating a domain package; not proof that packages follow it.
Absent	Required by the product vision but not exposed by the current implementation.
Unmeasured	No evidence yet establishes the claimed range, usability, or outcome.

These labels appear at the point of use because a single section may contain an implemented mechanism, a proposed extension point, and an unmeasured product outcome.

1. Coding is cheap; justified reliance is not

The important change in agentic software development is not merely that agents can generate whole features. It is that code production stops being the main constraint. More candidate implementations mean more claims to evaluate, more checks to run, more exceptions to understand, and more repairs whose effects have to survive beyond the session that made them.

The central product question becomes:

What would justify relying on this change, in this domain, under these conditions—and what would make that justification expire?

That is not a scalar question about whether an agent is trustworthy. A database query, an authentication replay, a UI accessibility claim, and a release rollback each require different semantics and different observations. Any generic system that answers them with one confidence score has already hidden the difficult part: who defined the claim, what was actually checked, what the check omitted, and which policy turned the result into permission.

TanStack Trust therefore treats trust as **domain-embedded work**. Its product value is reusable machinery that helps a domain make its grounds for reliance explicit, inspectable, renewable, and repairable. It should help an agent ask better questions and carry evidence forward. It must not let an agent manufacture authority by emitting a pass-shaped payload.

The adjacent AIUC argument is useful here. It separates raw capability from deployability, emphasizes bounded promises over unlimited guarantees, and asks whether controls have evidence of effectiveness and remain current. TanStack Trust adopts that problem analysis. It does not adopt certification, insurance, an independent-auditor institution, or a universal trust product. Trust remains deeply located in a software domain; the reusable layer serves that domain.

The first domain is Endpoints, and it will remain the product home for Trust while this architecture develops. The near-term question is whether these mechanics make Endpoints claims more inspectable and maintainable for agents and humans. It is not whether Trust can already justify a standalone product.

Status: The cheap-code/expensive-reliance premise is the product motivation selected for this RFC. It is not a measured claim about TanStack users, agent adoption, or improved software outcomes. **Unmeasured.**

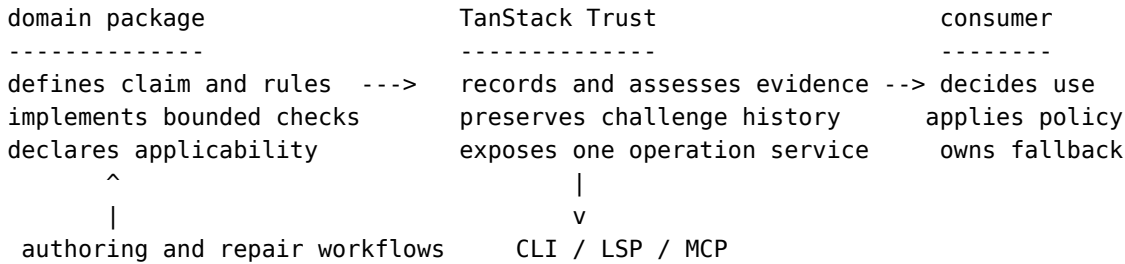
2. The authority map

The design has three semantic owners and two carriers:

Layer	Owns	Must not silently own
Domain package	Claim meaning, laws, check authority, evidence adequacy, applicability semantics, formal encodings, declared omissions	Generic transport or consumer permission
TanStack Trust	Evidence records, argument traversal, provenance, freshness, challenges, replay and repair history, shared operations	Domain truth, checker soundness, severity, or deployment action
Consumer policy	Permission, review requirements, severity, fallback, rollout and enforcement	Rewriting evidence state or erasing uncertainty
Workflow	Procedures for designing checks, maintaining evidence, and repairing failures	Granting evidence authority because steps were followed

Layer	Owns	Must not silently own
Interface	Transporting the same operations through CLI, LSP, MCP, or another adapter	Changing semantics because a result appears in an editor or agent tool

The basic handoff is:



Workflows and interfaces carry obligations across the handoff. They do not add a fourth source of semantic truth.

This is an authority map, not a package map. The five layers may share one repository, release, process, and user-facing product. The separation matters because it prevents Endpoints domain rules, generic evidence bookkeeping, and runtime policy from silently authorizing one another. It does not require an independent @tanstack/trust package.

The six base primitives

The exploratory Design Grammar identified six candidate building blocks. The current kernel represents most of them directly, although its applicability model is narrower than the grammar.

Primitive	Job
Claim	A versioned predicate about a subject under explicit conditions. It can exist before evidence.
Observation	An immutable reported result tied to one execution, method, origin, and captured context.
Rule	A registered, versioned domain procedure declaring the premises and side conditions it accepts.
Argument	A proposed application of a rule to observations or premise arguments for a claim.
Applicability condition	The domain relation between captured evidence context and a requested use.
Challenge	A targeted objection or observed counterexample with unresolved and resolution history.

A run is shared provenance for observations. A task is a projection of a gap. Neither becomes another source of authority.

Three axes, not one verdict

The system must report three independent questions:

Axis	Example states	Owner
Evidence	supported, contradicted, unresolved; no observation versus an inconclusive observation	Domain rules assessed by TanStack Trust
Execution and reach	check reached its declared production checkpoint; did not run; threw; reached only a substitute path	Check runner and domain contract

Axis	Example states	Owner
Policy	permitted, denied, requires review, fallback selected	Consumer

A solver returning unknown, a check failing to reach production behavior, an applicable counterexample, and a consumer denying an optimization are four different facts. Collapsing them to red, yellow, and green would make the system easier to display and harder to trust.

Status: This authority map is a **proposed contract**. The local Endpoints prototype implements much of the Trust column and keeps consumer policy out. It does not yet expose the stable internal extension boundary that a future extraction would require.

3. The domain package owns the promise

TanStack Trust can maintain evidence only after a domain has made its semantic commitments explicit. A complete domain package needs to declare at least:

1. the law, subject, conditions, and bounded scope of each claim;
2. one or more evidence routes, including which premises are jointly required and which routes are alternatives;
3. the registered rule and checker versions authorized to interpret evidence;
4. the reference or oracle, the production path, and the exact observation checkpoint;
5. the code, data, configuration, environment, method, and external inputs that can change applicability;
6. known omissions and unsupported constructs;
7. reach and fault controls that demonstrate the intended path and checker sensitivity; and
8. the replay obligation for a known failure.

The package should state a satisfying case, a violating case, and an unresolved case before implementing the happy path. This forces the package to distinguish application failure from check failure. A checker execution can succeed while producing a `fail` finding because it correctly observed an application violation; a checker that never reached the application has produced no correctness observation at all.

Evidence routes are domain choices

Several check forms fit the base. They are different access methods, not a universal strength ladder.

Form	What a domain must retain	What it cannot claim automatically
Deterministic certificate	Analyzed semantics, certificate contents, verifier, unsupported constructs, trusted components	Completeness or soundness of the analyzer and verifier
Empirical or oracle check	Generated histories, independent expected result, exercised production boundary, observations, seeds/schedules, replay and reduction	Behavior outside the campaign or unobserved environment
Rubric-assessed inspection	Exact source passages and representation, rubric, assessor provenance, conclusions and limits	Deterministic proof or independence merely because a review occurred
Formal model or solver check	Model or encoding, property, assumptions, scope/bound, tool configuration, witness, model-to-code claim	Truth of the encoding or correspondence with the implementation

For example, Datalog may expose which facts and rules derive a conclusion; Z3 may produce a satisfying model, an unsatisfiable core, or unknown; TLA+ and Alloy may produce a counterexample trace under a declared model and bound. Those outputs can become typed observations and explanations. The domain still owns the translation from software to facts, constraints, states, and properties.

Applicability is part of meaning

Evidence gathered under one context is useful only when the domain can say how that context relates to the requested use. Exact byte identity is sometimes an acceptable conservative boundary. It is not a general theory of semantic equivalence.

The Design Grammar describes applicability as a rule-owned relation with an explicit unknown result. The current kernel uses a narrower private boolean comparison over explicitly captured dependency revisions. These are two separate unsettled questions:

- Does the domain own the entire matcher, or does the base own a structural layer and call domain hooks for semantic comparison?
- Is applicability boolean, or must it distinguish applicable, inapplicable, and unknown?

The RFC does not answer those questions by hiding the difference. **Proposed contract; unresolved semantics.**

Soundness remains outside the base

Registered rule code is a trust root. The base can establish that a named rule was registered, that its declared premises were traversed, and that cited observations were current under the implemented matcher. It cannot establish that the rule encoded the right domain law, that a SQL analyzer found every effect, that a rubric is well chosen, or that a model corresponds to production.

This is the most important refusal in the architecture: the base never upgrades syntactic conformance to semantic authority.

Status: One Endpoints-owned package instantiates a `CheckContract` and a registered rule. Generic internal package authoring, rule inspection and registration, rubric admission, and formal-check integration are **absent**. They can first be developed for Endpoints without committing to a standalone Trust distribution.

4. The base owns evidence mechanics, not domain truth

Once a package has declared its semantics, substantial machinery becomes reusable.

Preserve the shape of support

Suppose claim C has two acceptable routes:

```
route A: C <- P1 AND P2
route B: C <- P3
```

P1 cannot replace P2, however strong its evidence looks. P3 is an alternative to the entire first route, not another item in one flat checklist. TanStack Trust therefore keeps premises conjunctive within an argument route and keeps routes disjunctive across arguments for the same claim. A flat gap list is only a deduplicated inventory; it is not a completion plan.

Cycles do not create grounding. A supported by B and B supported by A remains unresolved unless some route reaches independently admitted evidence. This finite grounding rule is **implemented** and covered by bounded tests.

Keep observation separate from interpretation

One check run may report several observations. Agreement, work performed, latency, and response value are different laws even when recorded together. Each observation retains its own claim, outcome, producer, check, captured case, and dependency context. A green aggregate or trusted origin does not grant an unmeasured claim.

The current runner uses three outcomes: pass, fail, and unresolved. Unknown analysis remains unresolved; it is not fabricated into a counterexample. If a check throws or fails to reach its declared checkpoint, the operation fails outside evidence history rather than manufacturing an observation. The failure can remain in CI or operational logs, but that separate history is not specified by the current Trust model. **Implemented evidence boundary; separate operational history unresolved.**

Make current authority explicit

Dependencies are named as code, data, configuration, environment, method, or external inputs. Each fingerprint change advances a monotonic revision. If a dependency changes from a to b and later back to a, the old evidence does not revive. The bytes may match; the old execution has not happened again in the current history.

The current rebuild invalidates only observations that captured a changed dependency. That is more precise than the earlier prototype's coarse global epoch, but it still depends on callers naming every relevant dependency. Recorded context is evidence of what was captured, not proof that context capture was complete. **Implemented mechanism; incomplete discovery.**

Retain contradiction and repair history

A failing observation opens a challenge. A newer unrelated pass cannot erase it. Resolution must link the original challenge to an applicable replay of the same law and case.

Timing is causal, not cosmetic. The replay check must be invoked after the failure is recorded. A pass that started earlier does not become repair evidence because its response arrived later. A second observation returned in the same failing batch is also insufficient. The trusted checker must execute fresh measurements rather than return cached data.

Resolution is append-only history plus current applicability. If the replay's dependencies later change, its current authority can expire and the original challenge becomes relevant again. The resolution record remains; the system does not rewrite history to pretend it never worked.

Exact same-case replay is deliberately conservative. Alternate-strategy repair, cross-version case mapping, and selective resolution expiry require explicit future semantics. **Implemented bounded repair; richer repair unresolved.**

Persist without overstating integrity

The current store validates counters and internal references, wraps state in a package-bound SHA-256 checksum, and writes through a temporary file followed by atomic rename. Requests are serialized inside one service process.

This protects against partial writes and detects accidental or out-of-band modification when the actor cannot also rewrite the checksum. It is not a signature, authentication system, hostile-plugin boundary, or multi-process transaction protocol. Two independent writers can still lose updates. **Implemented locally; security and multi-writer durability absent.**

Explanation is part of the mechanism

A useful explanation surface should expose:

- the successful route or unresolved routes;
- jointly required premises and alternative ways to satisfy them;
- observations, rules, runs, and semantic dependencies on each path;

- open challenge IDs and the evidence they defeat;
- applicability decisions and unknown reasons;
- check bounds, omissions, and counterexample witnesses; and
- shared dependencies without converting their count into confidence.

Datalog proof trees, SMT models and cores, model-checker traces, and provenance expressions can enrich this view when their domain packages use them. No one formal language is required for the base.

Status: The kernel retains structured routes and challenges internally. The full generic explanation contract is **proposed** and not a complete public operation.

5. Workflows carry maintenance obligations

Evidence becomes useful to later agents only if the project preserves how it was established and what must happen when it fails. The prototype packages three separate workflows.

Design a check

The design workflow starts with one proposed claim, its consumer, available source or code, and the consequence of a wrong conclusion. It requires authors to:

- state the law and its satisfying, violating, and unresolved cases;
- find boundaries with matched cases that vary one premise at a time;
- select an evidence route suited to the property;
- attack false acceptance and unnecessary rejection;
- verify that diagnostics distinguish alternative routes from joint work;
- run fault controls against the implementation under test; and
- package the rule versions, trust assumptions, omissions, and open decisions.

This follows the Oracle Guide's larger discipline: choose a law, generate or select histories that can reach it, assign fixture responsibility, and observe the production checkpoint. It also requires authors to distinguish harness failure from application findings, preserve the original violation while reducing a trace, and retain promises the bounded check did not establish.

Establish and maintain evidence

The maintenance workflow begins from the claim's structured routes, gaps, and challenges. An agent chooses a route for one missing premise, states what it can establish, captures relevant dependencies, runs the check, proposes an argument, and reassesses the original claim. Partial evidence leaves the broader obligation visible. A warning or permitted action is never recycled as evidence that the claim is established.

On a context change, the agent gathers new evidence only for affected uses. On a counterexample, it enters the repair workflow rather than adding unrelated green results.

Repair a claim or checker

Repair preserves the original law, inputs, context, observation, and rule versions. It first classifies the report: speculative objection, application bug, checker bug, generator gap, or operational failure. It then reproduces and reduces the failure while retaining the same violation, repairs the correct layer, reruns the original case and broader controls, and links only the exact applicable replay as resolution.

A narrower claim does not delete the original obligation. A different strategy gets its own argument. A solver scope reset, new model, or later green status is not repair history by itself.

What remains to prove about the workflows

The workflows are documented procedures. No agent study has yet established that they reduce mistakes, produce useful diagnostics, shorten correction cycles, or resist self-certification. The source material suggests matched evaluations in which fresh agents attempt a task, receive a diagnostic, and try to correct the problem under a fixed budget. That evaluation has not run.

Status: Workflow cards are **implemented as documentation**. Their agent ergonomics and effectiveness are **unmeasured**; contextual diagnostic rendering and correction evaluation are **absent**.

6. Interfaces expose one service unevenly

The prototype has one serialized operation service and three adapters. This is valuable because a CLI command, editor diagnostic, and agent tool do not grow different evidence semantics. It is path consistency, not independent corroboration.

Current semantic capability matrix

Capability	CLI	LSP	MCP	Status
Initialize a verified store	init	Via generic evidence.request	Not advertised	Implemented
Summarize store and open challenges	status	Via generic request	evidence_status	Implemented
Assess one exact structured claim	assess	Document diagnostics and generic request	evidence_assess	Implemented
Run the Endpoints safe-skip check	run-endpoints	Via generic request	evidence_run_endpoints	Implemented, Endpoints-specific
Update dependency context	context	Via generic request	evidence_context	Implemented
Read observations and challenge history	history	Via generic request	evidence_history	Implemented
Link a challenge to a replay	resolve	Via generic request	evidence_resolve	Implemented
Publish supported/contradicted/unresolved claim diagnostics	No	Yes	No	Implemented adapter behavior
Discover and inspect domain packages and rules	No	No	No	Absent
Author or register a versioned rule/check package	No	No	No	Absent
Propose a generic argument with typed citations	No	No	No	Absent
Ingest generic observations or rubric inspections	No	No	No	Absent
Request a stable, fully typed route explanation	Partial opaque assessment	Partial diagnostic data	Partial structured result	Proposed contract
Derive and manage gap-based agent work	No	No	No	Absent

The CLI accepts inline JSON, @FILE, or stdin and reports top-level failures with a nonzero status. The LSP reads JSON documents containing a claim, publishes no diagnostic for support, an error for contradiction, and a warning for unresolved evidence. That severity mapping is current adapter behavior, not a universal policy. It also exposes the shared service through `evidence.request`. The MCP server advertises six tools and returns both text and structured results.

All three use the same service and local store. They have been exercised by a shared integration test. No packaged editor configuration, MCP installation, schema-quality pass, real agent registration, or cross-process deployment has been demonstrated. **Implemented local adapters; distribution and actual agent use unmeasured.**

The Endpoints agent interface still missing

For agents to construct and maintain Endpoints guarantees rather than merely query one fixed check, the operation layer needs at least:

- package discovery and version inspection;
- rule/check registration through an authorized review boundary;
- typed claim, observation, argument, applicability, and challenge schemas;
- argument proposal without self-authorizing the proposed rule;
- generic check execution and honestly labeled inspection ingestion;
- complete explanation and dependency-path queries;
- gap and dependent-use queries suitable for agent work; and
- explicit capability/version negotiation across CLI, LSP, and MCP.

These operations can remain Endpoints-owned while their evidence semantics stay cleanly separated. Adding them is more important than extracting a package or adding another transport. It is the difference between “an agent can call the demo” and “an agent can participate in Endpoints’ evidence lifecycle.”

7. Consumer policy owns permission and fallback

An evidence assessment answers what current registered routes establish. It does not answer whether an application should deploy a change, skip a refresh, require a reviewer, or tolerate missing evidence.

This separation prevents two opposite errors:

1. treating an unresolved claim as a universal veto; and
2. treating the absence of a counterexample as permission.

A project may choose strict policy for a destructive migration and permissive policy for a reversible local optimization. It may treat a relevant observed contradiction more strongly than missing evidence. Those choices belong to the consumer because they depend on stakes, reversibility, operating environment, and organizational authority.

Fallback is also domain-specific. “Full refetch” sounds conservative in an Endpoints discussion, but it is not universally safe when effects are still in flight or external systems remain unobserved. The base should report the evidence and execution state that policy needs. It should not smuggle a fallback into the meaning of unresolved.

Status: The prototype correctly stops before enforcement. Universal severity, strictness, confidence aggregation, priority ordering, permission, and fallback are intentionally **unselected**.

8. Endpoints is the incubation host and bounded reconstruction

TanStack DB Endpoints supplies the first production-shaped domain package: endpoints/safe-skip-refetch@1.

It is also where Trust will live for now. Endpoints owns the complete product experience: its domain rules and checks, the internal Trust evidence mechanics, the agent-facing interfaces, and the consumer policy that will eventually act on assessments. Keeping those responsibilities separate inside Endpoints makes their contracts inspectable without forcing premature package extraction.

The claim concerns one retained query and one mutation analyzed under a shared build artifact. A refetch may be reported as safe to skip only when:

- the query and mutation belong to the same artifact;
- the collection has a confirmed authority baseline;
- no optimistic or repair obligation is pending;
- both effect bounds are complete; and
- the query’s read set is disjoint from the mutation’s write set.

External writes are outside the law.

The domain handoff is visible in three cases:

Case	Domain verdict	Evidence observation	What it does not decide
Complete disjoint bounds	skip	pass, capable of supporting the exact claim	Whether product policy should enable the optimization
Overlapping bounds or missing authority	refresh	fail, opening a challenge for the exact use	Whether another strategy could satisfy the broader requirement
Incomplete effects or different artifacts	unresolved	unresolved, no fabricated counterexample	Whether missing evidence should warn, block, or trigger other work

The fixture supplies pre-analyzed query and mutation summaries plus authority state. The extracted production helper canSkipRefetch computes only the bounded verdict over those inputs. The package’s CheckContract records the law, source, domain, trusted analyzer components, production path, checkpoint, observations, omissions, reach witness, fault controls, and replay method. A registered rule admits only a current skip observation from that exact check and producer.

An independent oracle enumerates small read/write subsets over three table names and computes intersection without calling the production decision helper. This tests the set-intersection decision kernel. It does not test whether the SQL analyzer found every table, whether PostgreSQL returned the right values, whether an external writer changed data, or whether skipping a refresh is the right production policy.

An omitted external writer illustrates the formal boundary. A solver may prove that the declared read and write sets are disjoint. If the model omitted a writer that can affect the query, the closed-world result does not establish the open-world production claim. The solver did its job; the domain encoding and context capture were incomplete.

At the inspected snapshot, this vertical has **bounded evidence** from 21 local tests and eight mutation controls across the kernel and rebuild. The rebuild is runnable working-tree state, not a committed or released package. Its SQL analyzer range, production consumer enforcement, external-write behavior, and generality remain outside the result.

Endpoints is not range evidence for TanStack Trust. It helped form the grammar and is therefore a constructive example, not an independent test that the base spans other domains. That limitation remains visible, but it does not create a near-term requirement to build another implementation.

9. Borrowed mechanisms, breakpoints, and decisions

TanStack Trust is an integration of useful mechanisms, not a claim to have invented proof, provenance, testing, or argumentation.

Borrowed-mechanism ledger

Donor	Mechanism carried into this design	Evidence status	Breakpoint / prohibited inference
Lean / proof-assistant analogy	Separate a small checking base from domain-specific rules and proof construction	Conceptual source analogy	Registered JavaScript rule code is still trusted; the prototype is not a theorem prover or Lean-like trusted kernel
ATMS and Carneades	Preserve alternative support, joint premises, defeaters, and inspectable argument structure	Source-reported design donor	Static argument structures do not supply current applicability, execution reach, or repair causality
Datalog / Souffle	Facts and recursive rules; proof trees showing producing rules and premises	Sighted primary-source donor pass	Entailment is relative to supplied facts and rules; positive derivation does not model applicability, defeat, expiry, or repair
SAT/SMT / Z3	Models as witnesses, unsatisfiable cores as sufficient conflicts, explicit unknown	Sighted primary-source donor pass	The domain encoding is a trust root; cores need not be minimal; solver state is not durable evidence history
TLA+ / Alloy	Properties over declared models; examples and counterexample traces under explicit state or finite bounds	Sighted primary-source donor pass	A model property is not automatically an implementation property; absence of a bounded counterexample is not an unbounded guarantee
Provenance semirings	Addition for alternative derivations, multiplication for joint use, expressions that expose shared inputs	Sighted paper and negative-control pass	Positive provenance has no semantics-free account of negation/difference; coefficients are not confidence or independent votes
Field Lab	Separate design, evidence maintenance, hostile testing, repair, provenance, and user checkpoints	Direct workflow lineage	Following a workflow does not grant domain authority or validate the result
shadcn / lint-style tooling	Contextual diagnostics, editable targets, and a correction loop close to code	Design transfer; evaluation not run	A plausible fix message is not evidence that agents can repair the problem or that the diagnostic is valid
Beads-style task interfaces	Project gaps and dependencies into work later agents can discover	Design transfer; generic operations absent	Task closure, notes, or counts do not grant support and may hide partial obligations
AIUC	Capability versus deployability, bounded promises, tested-control disclosure, and trust maintenance	Supplied-source transfer with user correction	No certification, insurance, universal standard, independent auditor, or overarching trust-product authority transfers
Endpoints	A concrete domain package exercising claims, checks, context, contradiction, replay, storage, and three interfaces	Bounded implementation	It is not independent generality evidence and does not establish analyzer or production-policy soundness

No donor justifies putting a generic solver in the base. The reusable abstraction is the evidence lifecycle and its authority boundaries, not one formal logic.

Decision register

The source trail leaves these boundaries unresolved:

Semantics

- **Applicability:** domain-owned matcher, layered structural/domain matcher, or another split; boolean versus three-valued result.
- **Identity:** semantic equivalence, scope subsumption, and cross-version claim or counterexample mapping beyond exact canonical JSON identity.

- **Challenges:** whether speculative objections become first-class objects distinct from observed failing challenges.
- **Repair:** alternate strategies, narrower claims, durable repair certificates, selective expiry, and distributed execution clocks.
- **Aggregation:** no universal confidence score, evidence-strength order, or total investigation-priority order is selected.

Domain evidence

- Admission and review rules for registered checkers, analyzers, rubrics, models, and solver encodings.
- Complete semantic-dependency capture and honest treatment of hidden inputs.
- Model-to-code correspondence and the range of formal checks.
- A machine-readable failure object containing environment, schedule, expected and actual results, original and reduced traces, reproduction class, and cleanup diagnostics.

Public product and interfaces

- A stable internal domain-package manifest and extension API that could support later extraction without requiring it now.
- Generic rule registration, argument proposal, check execution, observation ingestion, source inspection, explanation, and gap operations.
- Governance that lets agents propose rules without authorizing their own trust roots.
- Installed LSP and MCP clients, distribution, schema quality, capability negotiation, and real agent workflows.
- The boundary between evidence diagnostics and consumer severity or action.
- The conditions, ownership needs, or repeated friction that would justify extracting Trust from Endpoints into its own release boundary.

Infrastructure and security

- A single-daemon or coordinated multi-process storage boundary.
- Transactional semantics between dependency-context updates and check reach.
- Signatures, authentication, remote authority, hostile-plugin isolation, and evidence stores controlled by another actor.
- Scale behavior for large argument graphs, histories, recursive explanations, and long-lived projects.

Validation

- Broader SQL-analyzer and PostgreSQL evidence for Endpoints.
- Independent range remains unmeasured; no second domain implementation is currently planned. A future extraction decision may create a reason to test another domain then.
- Agent studies for check authoring, evidence maintenance, repair, diagnostic correction, and self-certification failure.
- Human reader and maintainer evaluation of the architecture and interfaces.
- Measured effects on software correctness, review burden, maintenance cost, or deployment outcomes.

Proposed build sequence

The recorded gaps suggest an Endpoints-first sequence:

1. **Complete the Endpoints trust loop.** Connect Endpoints rules and checks, internal Trust evidence mechanics, consumer policy, and agent-facing use in one end-to-end product path.
2. **Keep the internal seams explicit.** Decide applicability's ownership and result shape; define typed claims, observations, rules, arguments, challenges, dependencies, and check contracts without requiring a separate package.

3. **Make explanation complete.** Expose routes, joint premises, alternatives, challenge effects, applicability decisions, check bounds, and shared dependencies through one stable Endpoints service result.
4. **Add the operations Endpoints agents need.** Internal package discovery, authorized rule registration, check execution, argument proposal, evidence ingestion, and gap/dependent-use queries should precede more transport work.
5. **Broaden and connect the real vertical.** Keep safe-skip-refetch bounded, expand analyzer evidence, scope dependencies by endpoint or artifact, and connect assessments to explicit Endpoints consumer policy without moving that policy into Trust.
6. **Ship and evaluate the agent experience.** Package the CLI, LSP, and MCP integrations; test actual editors and agents; and measure the full design/maintain/repair loop rather than adapter parity alone.
7. **Harden for the deployment Endpoints actually needs.** Add multi-process coordination, signatures, hostile-process isolation, remote execution, and scale work under explicit threat and deployment models.
8. **Revisit extraction only when its value is concrete.** Another use, release boundary, ownership need, or repeated friction may justify pulling Trust out. Until then, clean internal seams preserve the option without making it a milestone.

This ordering is proposed, not frozen product policy. Its governing principle is simple: extraction or another interface is not progress if Endpoints agents still cannot see or preserve the semantic obligations behind the result.

Conclusion

The durable opportunity for developer tools in the agent era is not to make code generation marginally easier. Models will continue getting better at producing code and using APIs. The lasting value is in the application behavior that shared implementations make more reliable, efficient, and maintainable—and in the responsibility those implementations remove without hiding what remains.

TanStack Trust gives that opportunity a specific architecture inside Endpoints. Endpoints domain code owns the promise. The Trust layer owns evidence mechanics. Workflows carry maintenance and repair obligations. Interfaces make one service available to humans and agents. Endpoints consumer policy owns permission and fallback. These can be separate responsibilities without being separate products.

The Endpoints-hosted system succeeds when a later agent can tell, without reconstructing the whole project from scratch:

- what claim matters;
- which routes could support it;
- what was actually observed;
- which context makes that evidence applicable;
- which contradictions remain open;
- what exact replay repaired an earlier failure;
- what the checks still do not establish; and
- who has authority to decide what happens next.

That is how completeness can remain the engineering goal without becoming a false claim of universal proof. TanStack Trust should be complete about the grounds, limits, and maintenance of a domain's guarantees—even when the honest answer is unresolved.